ECUACIONES DIFERENCIALES

UNIVERSIDAD ESTATAL DE MILAGRO
UNEMI
Evolución Académica

UNIDAD 4

SOLUCIÓN NUMÉRICA DE UNA ECUACIÓN DIFERENCIAL

TEMA 1

MÉTODOS NUMÉRICOS PARA ECUACIONES DIFERENCIALES ORDINARIAS

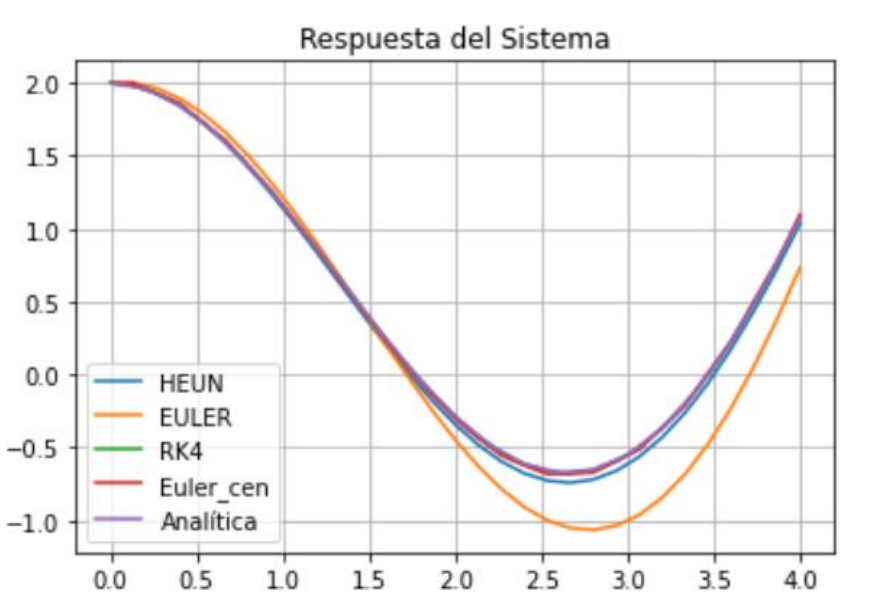


Figura 1: Métodos de solución numérica de una Ecuación Diferencial.
Fuente: Elaboración propia.

Ing. Jhonny Ortiz, Mgs

ACTIVIDAD DE INICIO

Acceda mediante el link o código QR

https://www.menti.com/wxixoqohzn

OBJETIVO

Resolver problemas con valor inicial de ecuaciones diferenciales ordinarias de primer hasta orden n y sistemas de ecuaciones de primer orden, aplicando métodos de Euler, Taylor y Runge Kutta.

SUBTEMAS

- **Subtema 1:** Método de Euler
- **Subtema 2:** Método de Taylor
- **Subtema 3:** Método de Runge Kutta 4 orden.
- **Subtema 4:** Sistemas de ecuaciones Diferenciales

INTRODUCCIÓN

Las ecuaciones diferenciales permiten expresar leyes de la naturaleza, se aplica en áreas como ingeniería, economía y ciencia aplicada.

Una ecuación diferencial, es una ecuación que contiene una variable dependiente y sus derivadas con respecto a una o más variables independientes.

$$m\frac{d^2y}{dt^2} = cy \quad \text{Ecuación de segundo orden}$$

$$(k - x^2)\frac{d^2z}{dx} - 2x\frac{dz}{dx} + q(q+1)z = 0 \quad \text{Ecuación de segundo orden}$$

$$\frac{dy}{dx} + kxy = e^{-x}\sin(x) \quad \text{Ecuación primer orden}$$

DESARROLLO

Las ecuaciones diferenciales que se resolverán son problemas de valor inicial.

Sea f: $\mathbb{R}\ x\ \mathbb{R}^n \longrightarrow \mathbb{R}$ una ecuación del siguiente tipo

$$f(t, x', x'', \ldots, x^{(n)}) = 0 \qquad (1)$$

Se llama ecuación diferencial de orden n.

Donde

$$x' = \frac{dx}{dt},\ x'' = \frac{d^2x}{dt^2},\ \ldots,\ x^{(n)} = \frac{d^nx}{dt^n}$$

Esta ecuación se define en el intervalo $a \le t \le b$ y está sujeta a las condiciones iniciales:

$$x(a) = \alpha_1,\ \ x'(a) = \alpha_2,\ \cdots,\ x^{n-1}(a) = \alpha_n$$

MÉTODO DE EULER

El método de Euler es la aproximación más básica, se basa en el cálculo de pendientes para extrapolar desde un valor anterior de x_i un nuevo valor de x_{i+1} en una distancia h. Esto se calcula paso a paso y de esa forma se traza una trayectoria de solución. (Chapra & Canale, 2006).

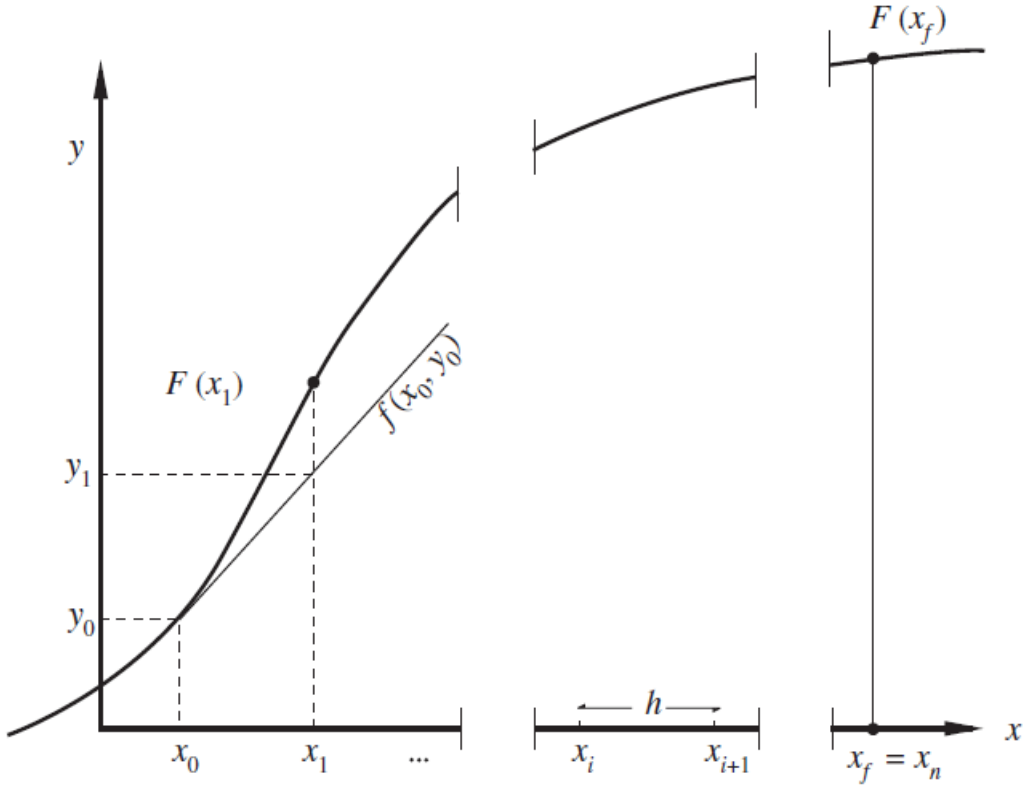


Figura 2: Deducción gráfica del método de Euler
Fuente: (Nieves & Dominguez, 2014)

MÉTODO DE EULER

Sea el problema de valor inicial

$$\begin{cases} x' = f(t,x) \\ x(t_0) = x_0 \end{cases}$$

La ecuación se puede discretizar utilizando la fórmula de diferenciación numérica:

$$x_i' = \frac{x_{i+1} - x_i}{h}$$

Donde

$$x_i = x(t_i)$$

$$t_{i+1} = t_i + h$$

Así el problema de valor inicial queda discretizado como:

$$x_{i+1} = x_i + hf(t_i, x_i) \qquad (2)$$

Código de Euler en Python

```
#METODO DE EULER
def Euler(T,X0,F,n):
  t=np.linspace(T[0],T[1],n+1)
  h=float(T[1]-T[0])/n
  X1=np.array(X0)
  X2=np.array([X1])
  for i in range(n):
    X1=X1+h*np.array(F(t[i],X1))
    X2=np.append(X2,[X1],axis=0)
  return t,X2
```

MÉTODO DE EULER

Ejercicio 1

Para el problema de valor inicial aplique Euler, halle la solución en x(1) y grafique la solución de la ecuación.

$$x' = t^2 - t + \sin(t)$$

$$x(0) = 1, \quad h = 0.2$$

Solución

$$h = \frac{t_n - t_0}{n}$$

$$n = \frac{1-0}{0.2} = 5$$

```
[4]  #solucion EULER ORDEN 1
     F=lambda t,x:t**2-t+np.sin(t)
```

```
[T1,X1]=Euler([0,1],[1],F,5)
np.set_printoptions(precision=5)
print(T1)
print(X1)
```

```
[0.  0.2 0.4 0.6 0.8 1. ]
[[1.     ]
 [1.     ]
 [1.00773]
 [1.03762]
 [1.10255]
 [1.21402]]
```

```
[38] plt.plot(T1,X1)
     plt.title('Respuesta del modelo')
     plt.grid()
     plt.show()
```

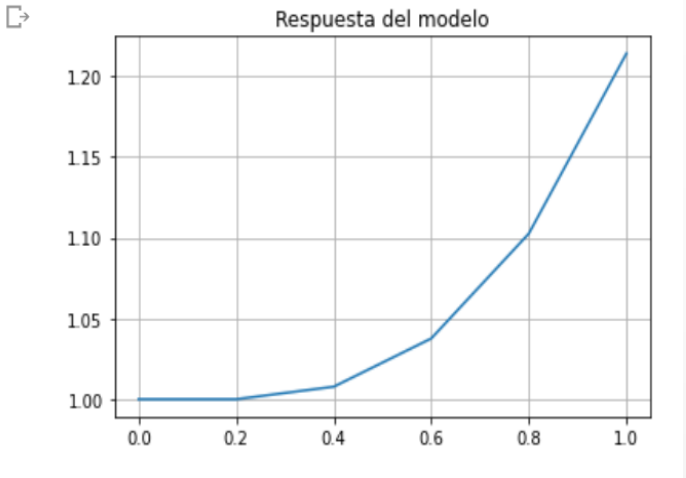


Figura 3: Código para Euler y gráfica de la solución

Fuente: Elaboración propia.

MÉTODO DE EULER

Ejercicio 2

Para el problema de valor inicial utilice Euler, halle la solución en y(2) y grafique la ecuación.

$$y' = -\frac{2}{t}y + t^2 e^t, \quad 1 \le t \le 2$$

$$y(1) = \sqrt{2}\mathrm{e}, \quad n = 100$$

Solución en Python

```
[48] #solucion EULER ORDEN 1
     F=lambda t,y:-(2/t)*y+(t**2)*np.exp(t)
```

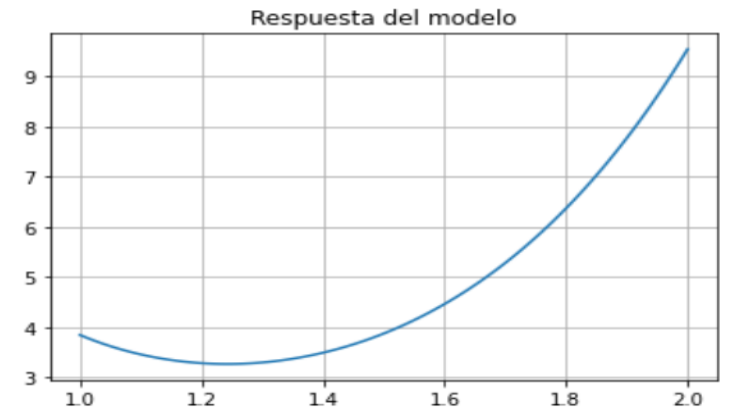
```
[51] [T1,X1]=Euler([1,2],[np.exp(1)*2**0.5],F,100)
     np.set_printoptions(precision=5)
     #print(T1)
     print(np.transpose(X1))
```

```
[[3.84423 3.79453 3.7474  3.70277 3.66059 3.62079 3.58333 3.54815 3.51521
  3.48446 3.45586 3.42938 3.40498 3.38262 3.36228 3.34393 3.32754 3.31309
  3.30056 3.28993 3.28119 3.27431 3.26929 3.26611 3.26476 3.26524 3.26753
  3.27164 3.27755 3.28526 3.29478 3.3061  3.31923 3.33416 3.35091 3.36947
  3.38985 3.41207 3.43612 3.46202 3.48978 3.5194  3.55091 3.58432 3.61964
  3.65689 3.69608 3.73724 3.78037 3.82551 3.87267 3.92187 3.97314 4.0265
  4.08197 4.13958 4.19936 4.26134 4.32553 4.39198 4.4607  4.53174 4.60512
  4.68088 4.75905 4.83967 4.92277 5.00838 5.09655 5.18732 5.28072 5.37679
  5.47557 5.57712 5.68146 5.78865 5.89873 6.01174 6.12774 6.24677 6.36888
  6.49413 6.62255 6.75421 6.88916 7.02746 7.16915 7.3143  7.46296 7.61519
  7.77106 7.93062 8.09394 8.26107 8.43209 8.60706 8.78605 8.96912 9.15635
  9.34781 9.54357]]
```

Figura 4: Código en Python y conjunto de valores solución ejercicio 2.

Fuente: Elaboración propia.

```
plt.plot(T1,X1)
plt.title('Respuesta del modelo')
plt.grid()
plt.show()
print(X1[-1])
```




```
[9.54357]
```

Figura 5: Respuesta del modelo por el Método de Euler

Fuente: Elaboración propia.

MÉTODO DE EULER

Método de Euler de diferencia centrada

Si utilizamos la fórmula de diferencia centrada

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} - \frac{h^2}{6} f''(c)$$

con $c \in [x-h, x+h]$

Se obtiene el método de Euler de diferencia centrada:

$$x_0 = x(t_0)$$

$$x_1 = x_0 + hf(t_0, x_0)$$

$$x_{i+1} = x_{i-1} + 2hf(t_i, x_i)$$

Código en Python

```
#METODO DE EULER CENTRADO
def Euler_cen(T,X0,F,n):
  t=np.linspace(T[0],T[1],n+1)
  h=float(T[1]-T[0])/n
  Xa=np.array(X0)
  X2=np.array([Xa])
  Xb=Xa+h*np.array(F(t[0],Xa))
  X2=np.append(X2,[Xb],axis=0)
  for i in range(n-1):
    X1=Xa+2*h*np.array(F(t[i+1],Xb))
    Xa=Xb
    Xb=X1
    X2=np.append(X2,[X1],axis=0)
  return t,X2
```

Ecuaciones de orden superior

Una ecuación diferencial de orden **orden 2** tiene la forma:

$$x'' = f(t, x, x')$$

Con las condiciones iniciales

$$x(t_0) = x_a \text{ y } x'(t_0) = x_b$$

La ecuación de segundo orden puede expresarse como un sistema de ecuaciones:

$$\begin{cases} x_0' = x_1 \\ x_1' = f(t, x_0, x_1) \end{cases}$$

MÉTODO DE EULER

Ejercicio 3

Utilice el método de Euler Centrado para obtener la solución de:

$$y'' - 0.5t + y = 0$$

Con valores iniciales $y(0) = 2 \text{ y } y'(0) = 0$, grafique la solución para valores de x=0 a 4, con h=0.1.

Realice las gráficas de la solución numérica y con la ecuación analítica cuya solución es:

$$y = 2\cos(t) - 0.5sen(t) + 0.5t$$

También calcule el error de ambas soluciones en x=4.

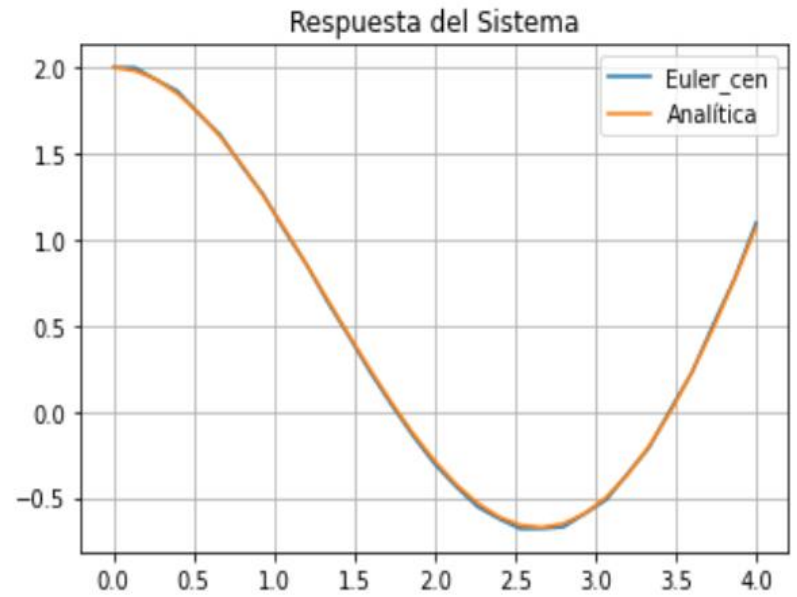
Solución

Se debe despejar la derivada más alta, en este caso es la de segundo orden, luego transformar en un sistema de ecuaciones.

$$y'' = 0.5t - y$$

$$y_0' = y_1$$
$$y_1' = 0.5t - y_0$$

```
F=lambda t,y:[y[1],0.5*t-y[0]]
[T1,Y1]=Euler_cen([0,4],[2,0],F,30)
ya=2*np.cos(T1)-0.5*np.sin(T1)+0.5*T1
plt.plot(T1,Y1[:,0],T1,ya)
plt.title('Respuesta del Sistema')
plt.legend(['Euler_cen','Analítica'])
plt.grid()
plt.show()
err_Eu=abs(ya[-1]-Y1[-1,0])
print(err_Eu)
```




```
0.02549863240028305
```

Figura 6: Código en Python y grafica de comparación solución analítica y numérica.

Fuente: Elaboración propia.

MÉTODO DE EULER

Método de Heun

Este método emplea dos derivadas para calcular el valor de y en el punto x_{i+1} (una derivada es en x_i y la otra en x_{i+1}) luego se promedian ambos valores para obtener una mejor estimación de la pendiente (Chapra & Canale, 2006).

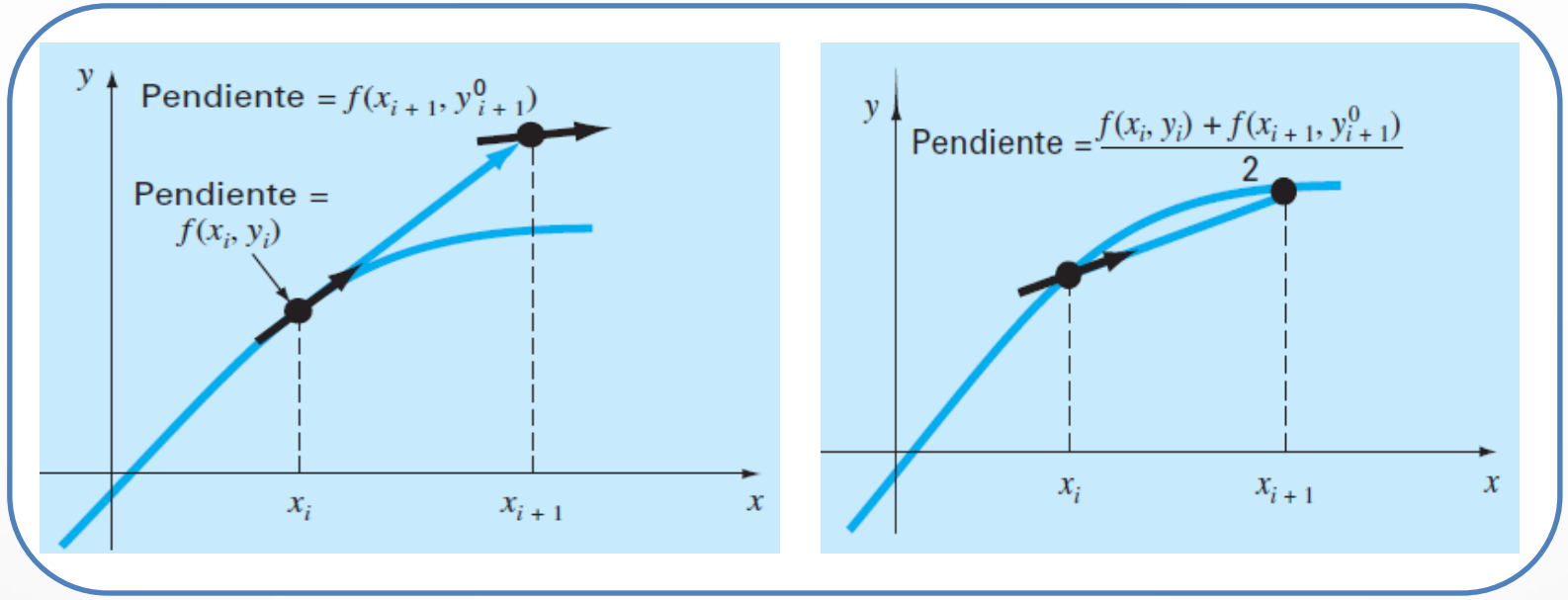


Figura 7: Representación gráfica del método de Heun (Predictor-Corrector).

Fuente: (Chapra & Canale, 2006)

MÉTODO DE EULER

Método de Heun

El método de Heun es un procedimiento predictor-corrector, consiste en realizar un promedio entre xi' e xi+1'

Así $\bar{x}' = \dfrac{x_i' + x_{i+1}'}{2}$ donde $x_i' = f(t_i, x_i)$,

$$x_{i+1}^* = x_{i-1} + hf(t_i, x_i)$$

$$x_{i+1}^* = x_i + hf(t_i, x_i) \quad \text{(Paso predictor)}$$

$$x_{i+1} = x_i + \frac{f(t_i, x_i) + f(t_{i+1}, x_{i+1}^*)}{2} \quad \text{(Paso corrector)}$$

Código en Python

```
def Heun(T,X0,F,n):
  t=np.linspace(T[0],T[1],n+1)
  h=float(T[1]-T[0])/n
  X1=np.array(X0)
  X2=np.array([X1])
  for i in range(n):
    w=np.array(F(t[i],X1))
    xa=X1+h*w #PASO PREDICTOR
    X1=X1+0.5*(w+np.array(F(t[i]
,xa)))*h #PASO CORRECTOR
    X2=np.append(X2,[X1],axis=0)
  return t,X2
```

MÉTODO DE HEUN

Ejercicio 4

Resuelva la siguiente ecuación diferencial de segundo orden aplicando el Método de HEUN

$$x'' = -\left(2t^2 - 3tx + x^2 + xsen(t)\right)x' - 5x^2 sen(t)$$

$$x(1) = 0.91\ ,\ \ x'(1) = 0.56$$

$$1 \le t \le 2.5$$

Solución

La ecuación de segundo orden se debe convertir en un sistema de ecuaciones

$$x_0' = x_1$$

$$x_1' = -\left(2t^2 - 3tx_0 + x_0^{\ 2} + x_0 sen(t)\right)x_1 - 5x_0^{\ 2} sen(t)$$

```python
##ECUACIÓN DE SEGUNDO ORDEN
F=lambda t,x:[x[1],-(2*t**2-3*t*x[0]+x[0]**2+x[0]*np.sin(t))*x[1]-5*x[0]**2*np.sin(t)]
```

```python
[T1,X1]=Heun([1,2.5],[0.91,0.56],F,100)
RR=[[T1],[X1[:,0]]]
X1[:,0]
```

```
array([0.91   , 0.91795, 0.92501, 0.93115, 0.93637, 0.94067, 0.94404,
       0.94648, 0.94799, 0.94858, 0.94825, 0.94702, 0.94489, 0.94187,
       0.938  , 0.93328, 0.92775, 0.92142, 0.91432, 0.90649, 0.89795,
       0.88875, 0.87891, 0.86848, 0.8575 , 0.846  , 0.83403, 0.82164,
       0.80887, 0.79576, 0.78236, 0.76872, 0.75488, 0.74089, 0.72679,
       0.71263, 0.69844, 0.68427, 0.67017, 0.65616, 0.64229, 0.62859,
       0.61508, 0.60181, 0.5888 , 0.57607, 0.56364, 0.55154, 0.53978,
       0.52838, 0.51735, 0.50669, 0.49642, 0.48654, 0.47705, 0.46796,
       0.45926, 0.45095, 0.44302, 0.43547, 0.42829, 0.42147, 0.41501,
       0.40889, 0.4031 , 0.39764, 0.39248, 0.38761, 0.38303, 0.37871,
       0.37466, 0.37085, 0.36726, 0.3639 , 0.36075, 0.35779, 0.35501,
       0.35241, 0.34996, 0.34767, 0.34552, 0.3435 , 0.34161, 0.33983,
       0.33816, 0.33659, 0.33512, 0.33373, 0.33242, 0.33118, 0.33002,
       0.32892, 0.32788, 0.32689, 0.32596, 0.32508, 0.32424, 0.32345,
       0.3227 , 0.32198, 0.3213 ])
```

```python
plt.plot(T1,X1[:,0])
plt.title('Respuesta del modelo')
plt.grid()
plt.show()
```

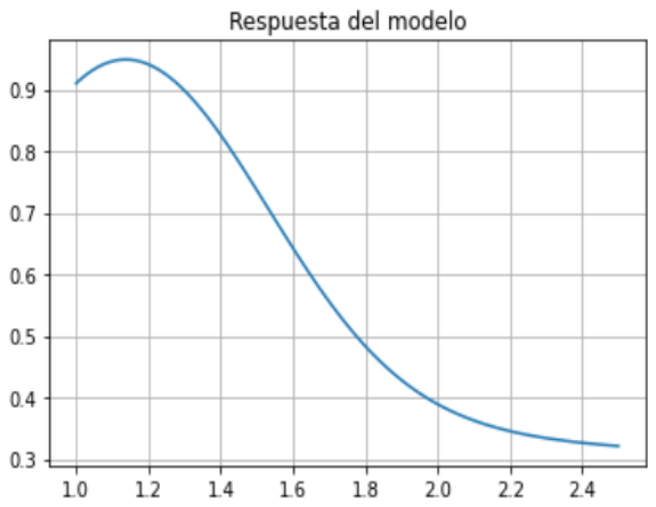


Figura 8: Código en Python Heun para ecuación de segundo orden y respuesta del modelo.

Fuente: Elaboración propia.

MÉTODO DE TAYLOR

Las técnicas numéricas tienen como objetivo determinar estimaciones precisas con el mínimo esfuerzo, para ello se requiere comparar la eficiencia de las aproximaciones mediante el error de truncamiento local. El error de truncamiento local, no solo sirve para determinar el error local de un método sino también el error de aproximación real (Burden & Faires, 2011).

$$x(t+h) = x(t) + hx'(t) + \frac{1}{2}h^2 x''(t) + \cdots + \frac{1}{k!}h^k x^{(k)}(t) + \frac{1}{(k+1)!}h^{k+1}x^{(k+1)}(c)$$

Taylor orden 2

Sea el problema de valor inicial

$$\begin{cases} x' = f(t,x) \\ x(t_0) = x_0 \end{cases}$$

La ecuación de Taylor orden 2 en términos de x se expresa mediante:

$$x_{i+1} = x_i + hx_i' + \frac{h^2}{2}x_i''$$

Para aplicar Taylor de orden 2 se necesita de la segunda derivada de x, representada por:

$$x''(t) = f_t(t,x) + f(t,x)x'(t)$$

La fórmula de Taylor de Orden 2 para una ecuación diferencial de primer orden es:

$$x_{i+1} = x_i + hf(t_i, x_i) + \frac{h^2}{2}\left(f_t(t,x) + f(t,x)x'(t)\right)$$

MÉTODO DE TAYLOR

Ejercicio 5

Aplique el método de Taylor orden 2 para resolver la siguiente ecuación:

$$y' = t^{-2}(\sin 2t - 2ty)$$
$$1 \le t \le 2$$
$$y(1) = 2 \quad \text{con } h = 0.1$$

Solución

Para aplicar Taylor orden 2 necesitamos y’’

$$y'' = -2t^{-3}(\sin 2t - 2ty) + t^{-2}(2\cos(2t) - 2y - 2ty')$$

$$y_{i+1} = y_i + hy_i' + \frac{h^2}{2} y_i''$$

```python
F1=lambda t,y:(t**-2)*(np.sin(2*t)-2*t*y)

F2=lambda t,y,ya:-2*(t**-3)*(np.sin(2*t)-2*t*y)+(t**-2)*(2*np.cos(2*t)-2*y-2*t*ya)

def Taylor(T,X0,F1,F2,n):
  t=np.linspace(T[0],T[1],n+1)
  h=float(T[1]-T[0])/n
  X1=np.array(X0)
  X2=np.array([X1])
  for i in range(n):
    w=np.array(F1(t[i],X1))
    w2=np.array(F2(t[i],X1,w))
    X1=X1+h*w+(0.5*h**2)*w2
    X2=np.append(X2,[X1],axis=0)
  return t,X2
```

```python
[T1,X1]=Taylor([1,2],[2],F1,F2,10)
X1
```

```
array([[2.     ],
       [1.72858],
       [1.50696],
       [1.32116],
       [1.1621 ],
       [1.02371],
       [0.9019 ],
       [0.79384],
       [0.69754],
       [0.61159],
       [0.53494]])
```

```python
plt.plot(T1,X1)
plt.title('Taylor orden 2')
plt.grid()
plt.show()
```

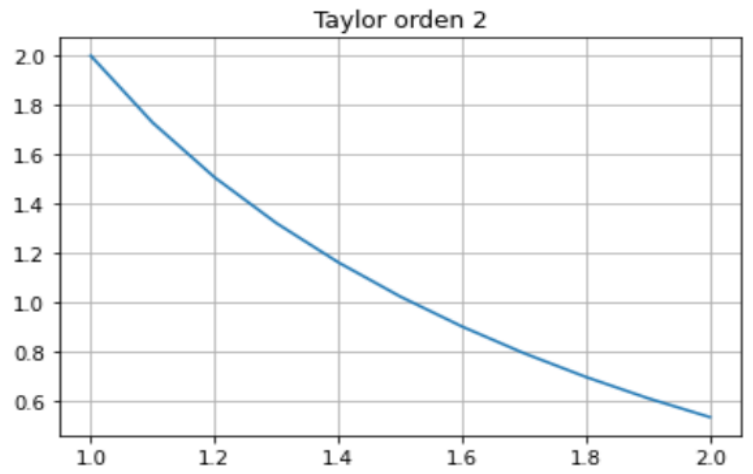


Figura 9: Código en Python método de Taylor orden 2 y respuesta del sistema.

Fuente: Elaboración propia.

MÉTODO DE RUNGE KUTTA 4 ORDEN

Los métodos de Runge-Kutta tienen error de truncamiento local superior a los métodos de Taylor y la ventaja es que no se requiere calcular y evaluar las derivadas de f(t,x).

Existen algunos métodos de Runge Kutta como el de segundo orden, tercer orden y cuarto orden, en esta sección se hará la descripción y aplicación del método de Runge Kutta cuarto orden.

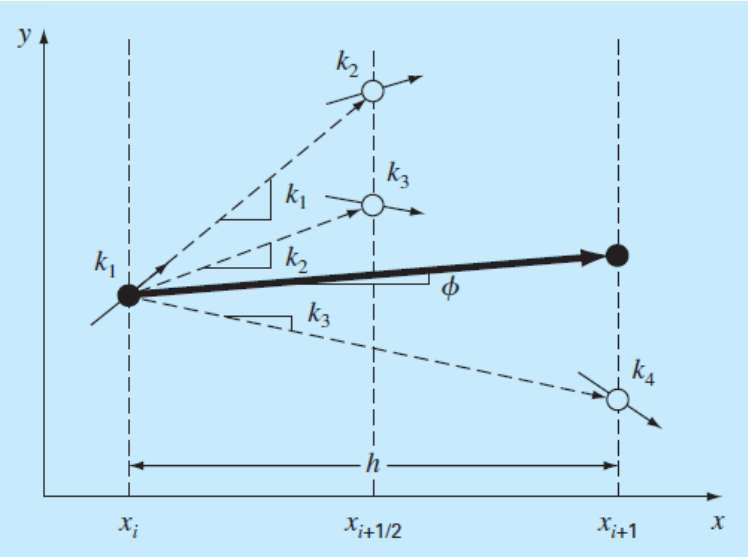


Figura 10: Método de Runge Kutta 4 orden.

Fuente: (Chapra & Canale, 2006).

MÉTODO DE RUNGE KUTTA 4 ORDEN

El método de Runge kutta cuarto orden tiene similitud con el método de Heun debido a que se utilizan múltiples estimaciones de la pendiente para obtener una mejor pendiente promedio (Chapra & Canale, 2006).

$$k_1 = hf\left(t_i, x_i\right)$$

$$k_2 = hf\left(t_i + \frac{1}{2}h, x_i + \frac{1}{2}k_1\right)$$

$$k_3 = hf\left(t_i + \frac{1}{2}h, x_i + \frac{1}{2}k_2\right)$$

$$k_4 = hf\left(t_i + h, x_i + k_3\right)$$

$$x_{i+1} = x_i + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)$$

Código en Python

```
def rk4(T,X0,F,n):
  t=np.linspace(T[0],T[1],n+1)
  h=float(T[1]-T[0])/n
  X1=np.array(X0)
  X2=np.array([X1])
  for i in range(n):
    k1=h*np.array(F(t[i],X1))
    k2=h*np.array(F(t[i]+0.5*h,X1+0.5*k1))
    k3=h*np.array(F(t[i]+0.5*h,X1+0.5*k2))
    k4=h*np.array(F(t[i]+h,X1+k3))
    X1=X1+(1/6)*(k1+2*k2+2*k3+k4)
    X2=np.append(X2,[X1],axis=0)
  return t,X2
```

MÉTODO DE RUNGE KUTTA 4 ORDEN

Ejercicio 6 **Fuente:** (Chapra & Canale, 2006)

Un sistema masa resorte está descrito por la ecuación diferencial:

$$m\frac{d^2x}{dt^2}+c\frac{dx}{dt}+kx=0$$

Siendo x el desplazamiento desde el equilibrio, m=25Kg la masa, c el coeficiente de amortiguamiento (N*s/m). Realice las gráficas para tres valores de c que son 5 (sub-amortiguado), 40 (amortiguamiento critico), 200 (sobre-amortiguado). La constante del resorte k = 20 N/m. El desplazamiento y la velocidad inicial son cero. Resuelva mediante Runge Kutta cuarto orden en el tiempo de 0 a 15 segundos, con los tres valores de c sobre la misma curva.

Solución:

```
m=20 #masa en Kg
c0=5 #coeficiente de amortiguamiento
k=20 #constante del resorte
F=lambda t,x:[x[1],-(c0/m)*x[1]-(k/m)*x[0]]
```

```
c1=40 #coeficiente de amortiguamiento
F1=lambda t,x:[x[1],-(c1/m)*x[1]-(k/m)*x[0]]
```

```
c2=200 #coeficiente de amortiguamiento
F2=lambda t,x:[x[1],-(c2/m)*x[1]-(k/m)*x[0]]
```

```
[T,X]=rk4([0,15],[1,0],F,100)
[T1,X1]=rk4([0,15],[1,0],F1,100)
[T2,X2]=rk4([0,15],[1,0],F2,100)
plt.plot(T,X[:,0],T1,X1[:,0],T2,X2[:,0])
plt.title('Respuesta masa resorte')
plt.grid()
plt.legend(['c=5 Subamortiguado','C=40 Amortiguamiento Crit','c=200 Sobre Amortiguado'])
plt.show()
```

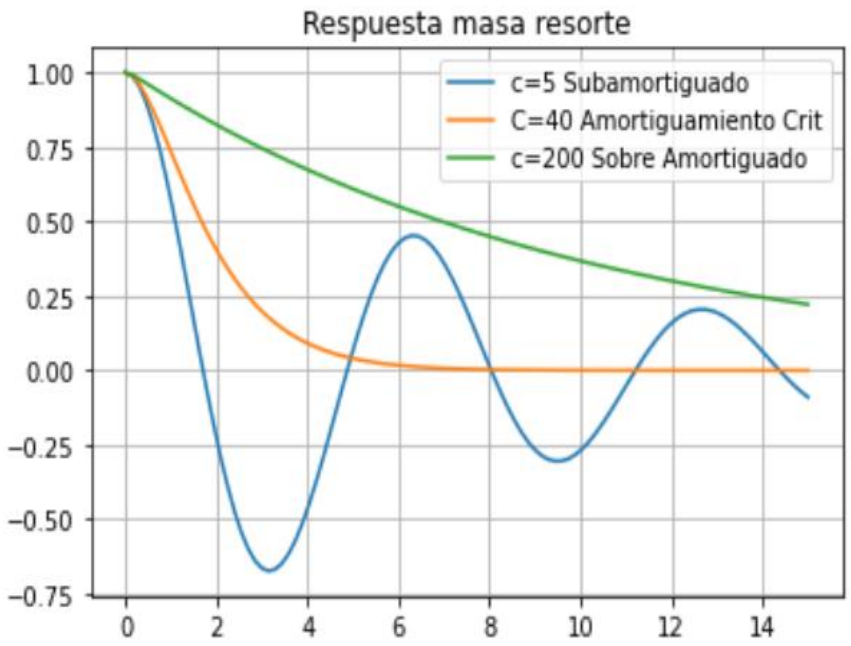


Figura 11: Respuesta del sistema masa-resorte ante variaciones de c.

Fuente: (Chapra & Canale, 2006).

SISTEMAS DE ECUACIONES DIFERENCIALES

Un sistema de ecuaciones diferenciales de primer orden se representa de la siguiente forma:

$$\begin{cases} x_1' = f_1(t, x_1, x_2, \ldots, x_n) \\ x_2' = f_2(t, x_1, x_2, \ldots, x_n) \\ \vdots \qquad\qquad \vdots \\ x_n' = f_n(t, x_1, x_2, \ldots, x_n) \end{cases}$$

Con las condiciones iniciales

$$x_1(t_0) = x_{10},\ x_2(t_0) = x_{20}, \cdots,\ x_n(t_0) = x_{n0}$$

Ejercicio 7

El sistema de ecuaciones para cinco reactores es el siguiente: Todas las concentraciones iniciales son cero. Halle la gráfica de solución de cada una de las concentraciones.

$$\frac{dc_1}{dt} = -0.12c_1 + 0.02c_3 + 1$$

$$\frac{dc_2}{dt} = 0.15c_1 - 0.15c_2$$

$$\frac{dc_3}{dt} = 0.025c_2 - 0.225c_3 + 4$$

$$\frac{dc_4}{dt} = 0.1c_3 - 0.1375c_4 + 0.025c_5$$

$$\frac{dc_5}{dt} = 0.03c_1 + 0.01c_2 - 0.04c_5$$

Solución en Python

```
F=lambda t,c:[-0.12*c[0]+0.02*c[2]+1,0.15*c[0]-
0.15*c[1],0.025*c[1]-0.225*c[2]+4,0.1*c[2]-
0.1375*c[3]+0.025*c[4],0.03*c[0]+0.01*c[1]-
0.04*c[4]]
[T,C]=rk4([0,120],[0,0,0,0,0],F,100)
plt.figure(figsize=(10,15))
plt.subplot(5,1,1)
plt.plot(T,C[:,0])
plt.legend(['C1'])
plt.grid()
plt.subplot(5,1,2)
plt.plot(T,C[:,1])
plt.legend(['C2'])
plt.grid()
plt.subplot(5,1,3)
plt.plot(T,C[:,2])
plt.legend(['C3'])
plt.grid()
plt.subplot(5,1,4)
plt.plot(T,C[:,3])
plt.legend(['C4'])
plt.grid()
plt.subplot(5,1,5)
plt.plot(T,C[:,4])
plt.legend(['C5'])
plt.grid()
plt.show()
```

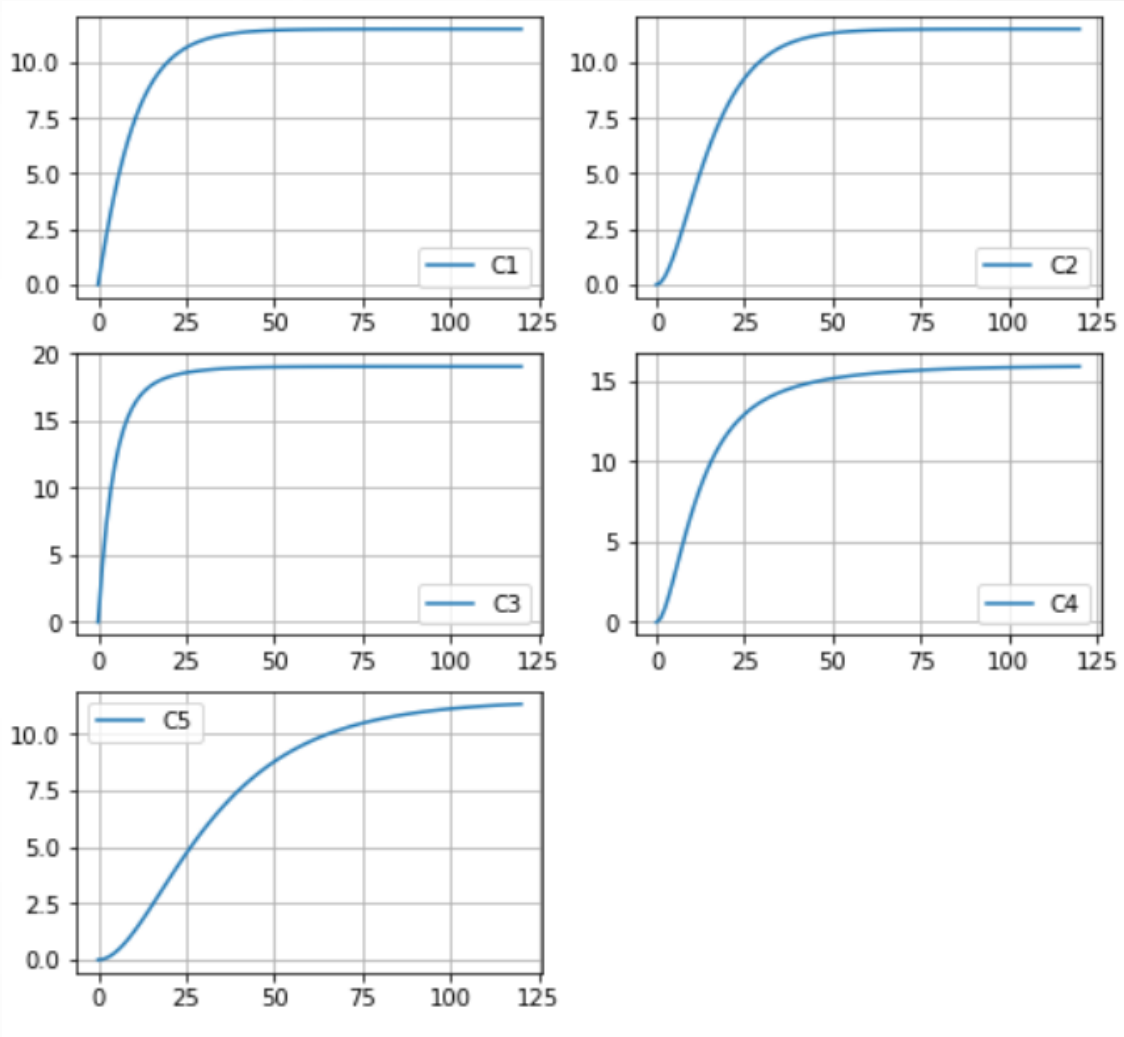


Figura 12: Respuesta de los reactores en función del tiempo.

Fuente: Elaboración propia.

SISTEMAS DE ECUACIONES DIFERENCIALES

Ejercicio 2

Resuelva mediante Heun, la siguiente ecuación diferencial de orden 3 con valores iniciales:

$$y''' - 2y'' + y' = xe^x + 5$$
$$y(0) = 2,\ y'(0) = 2,\ y''(0) = -1$$
$$0 \le x \le 0.5$$

Solución

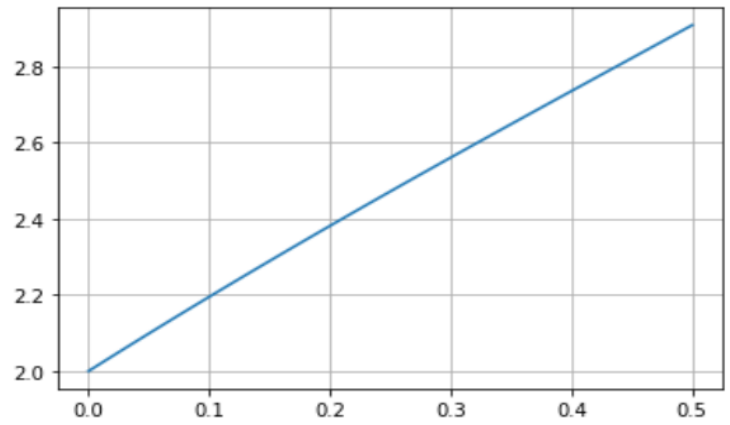
Se debe convertir a un sistema de ecuaciones que quedaría:

$$\begin{cases} y_0' = y_1 \\ y_1' = y_2 \\ y_2' = xe^x + 5 + 2y_2 - y_1 \end{cases}$$

```
F=lambda x,y:[y[1],y[2],x*np.exp(x)+5+2*y[2]-y[1]]
```

```
[T1,X1]=Heun([0,0.5],[2,2,-1],F,80)
X1[:,0]
```

```
plt.plot(T1,X1[:,0])
plt.grid()
plt.show()
print(X1[-1,0])
```




```
2.908893484951926
```

Figura 13: Código en Python y respuesta del modelo de tercer orden

Fuente: Elaboración propia.

PREGUNTAS DE COMPRENSIÓN

- ¿Qué tan buena es la estimación de la solución de una ecuación diferencial por el método de Euler?
- ¿En qué se basa el método de Heun?
- ¿Por qué no resulta practico aplicar el método de Taylor?
- ¿En qué se basa el Método de Runge Kutta cuarto orden?

BIBLIOGRAFÍA

- Burden, R., & Faires, D. (2011). Numerical Analysis. (BOOKS/COLE, Ed.) (Ninth). Boston.
- Chapra, S., & Canale, R. (2006). Métodos numéricos para ingenieros. (McGrawHill, Ed.) (Quinta). México.
- Nieves, A., & Dominguez, F. (2014). Métodos Numéricos aplicados a la ingeniería. (Patria, Ed.) (Primera). México.
- Sauer, T. (2012). Análisis Numérico. (PEARSON, Ed.) (Segunda). México. Retrieved from http://librosysolucionarios.net/